GLOSS tutorial — maintaining the server

How to push code changes to the ten participant instances, and what to do when something breaks. Contains no passwords.

What you need to know first

The two Docker images contain **no application code** — the Dockerfile copies only `environment_linux.yml`. Each participant's code lives in `/srv/gloss/pNN` on the host and is bind-mounted into their container at the same path.

> **So a code change needs no image rebuild and no container restart.** Copy the files in and they are live. Only a change to `environment_linux.yml` requires a rebuild.

Layout

```
~/gloss-repo          a git clone of the tutorial-deployment branch  (pull here)
~/gloss-clean         an rsync target, if you push from a laptop instead
/srv/gloss/_template  the golden copy all instances are synced from
/srv/gloss/p01..p10   one independent checkout per participant
/srv/gloss/_backups   files overwritten by an update, timestamped
/srv/gloss/images-backup.tgz   saved images, for disaster recovery
```

Workflow A — a collaborator pushes via GitHub (recommended)

Your collaborator clones the repo, commits to the branch, and pushes as normal:

```
git clone -b tutorial-deployment https://github.com/UbiWell/GLOSS.git
# ...edit, commit...
git push origin tutorial-deployment
```

Then on the server, pull and distribute:

```
ssh -p 65173 tutorial@207.56.9.25
cd ~/gloss-repo
git pull

# See what would change, without touching anything:
sudo bash deploy/update_instances.sh

# Distribute to all ten instances:
sudo bash deploy/update_instances.sh --apply
```

The dry run prints a per-participant list of the files that would change, so you can check the blast radius before applying. Changes take effect for the next query a participant runs; nothing needs restarting.

Workflow B — push straight from your laptop

Useful for work that is not committed yet. Run from your local checkout:

```
rsync -az --delete \
  --exclude '.git' --exclude '__pycache__' --exclude 'sample_data_old' \
  --exclude 'deploy/participants.txt' \
  --exclude 'deploy/GLOSS-tutorial-logins.*' \
  -e 'ssh -p 65173' \
  ./ tutorial@207.56.9.25:~/gloss-clean/

ssh -p 65173 tutorial@207.56.9.25 \
  'cd ~/gloss-clean && sudo bash deploy/update_instances.sh --apply'
```

> Always exclude `deploy/participants.txt`. It holds every participant's password; overwriting it with a stale copy would break logins, and committing it would publish them.

What the update does and does not touch

- A participant's edit to a file that *also* changed upstream is overwritten, with the old version kept in `/srv/gloss/_backups/pNN-<timestamp>/`.
- Files a participant created that do not exist upstream are left alone, unless you pass `--force`.
- Ownership is reset to the participant's uid *inside* the container (not the host account's uid), so their files stay editable.
- Credentials, generated handouts, `code_generation.py` and other run artifacts are never distributed.

When you must rebuild (environment changes only)

If `environment_linux.yml` changes, run the full provisioner. It rebuilds both images and recreates all ten containers, which takes roughly 30 minutes on a cold cache and **kills any session in progress**:

```
cd ~/gloss-repo
sudo -E env GATEWAY_API_KEY="$GATEWAY_API_KEY" PARTICIPANTS=10 \
  bash deploy/provision.sh
```

Passwords are preserved: the script reuses whatever is already in `deploy/participants.txt`. Run it from a directory that *has* that file, or it will generate new ones and invalidate your handout.

Reset one participant

For someone who has broken their copy. Takes about two seconds and does not disturb the other nine or require a restart:

```
sudo rsync -a --delete /srv/gloss/_template/ /srv/gloss/p03/
sudo chown -R 1000:1000 /srv/gloss/p03
```

If they have broken the *container* rather than the code:

```
sudo docker restart gloss-p03
```

Checking on things

```
sudo docker ps --filter name=gloss-p            # all ten should say "Up"
sudo docker logs gloss-p03 --tail 20            # why an instance won't start
sudo bash deploy/verify.sh                      # fast checks on every instance
sudo FULL=1 bash deploy/verify.sh               # ...plus a real query each (slow)
sudo systemctl list-timers gloss-janitor.timer  # container cleanup timer
df -h /                                         # disk
```

Things that will bite you

- **Never run `docker system prune -a`.** The executor image exists only on this host; pruning it breaks code generation for everyone at once, with a misleading "pull access denied" error. Recover with `sudo docker load < /srv/gloss/images-backup.tgz`.
- **Ports 22001–22010 are blocked** by the provider's firewall. Participants connect on port 65173 as `pNN`; their login shell drops them into their container. The published ports are only reachable from the host itself, which is what the browser-UI tunnel uses.
- **The model gateway allowlists this server's IP.** If every instance suddenly fails at ACTION PLAN GENERATION, check it first: `curl -s -o /dev/null -w '%{http_code}' -H "Authorization: Bearer $GATEWAY_API_KEY" https://compute-gateway.europa.khoury.northeastern.edu/api/tags`. An HTML 403 means the allowlisting lapsed; nothing local will fix it.
- **Participants have effective root on the host** via the mounted Docker socket. That was a deliberate trade for simplicity: keep nothing sensitive here and tear the server down afterwards.

Reprinting the participant handout

After any change to credentials or instructions:

```
python3 deploy/make_handout.py --host 207.56.9.25 --port 65173
```

That writes `deploy/GLOSS-tutorial-logins.pdf`, which contains all ten passwords — do not commit it or circulate it beyond the handout.